

Deploy an Image from AWS Elastic Container Registry (ECR) to AWS Elastic Kubernetes Service (EKS)

1. Prerequisites

Ensure the following are installed and configured:

- AWS CLI v2
- kubectl
- eksctl (optional but recommended)
- Docker

Ensure you have:

- An AWS Account
- An Amazon ECR repository
- A Docker image pushed to ECR
- An Amazon EKS cluster
- IAM permissions to access EKS and ECR

2. Configure AWS CLI

```
aws configure
```

```
AWS Access Key ID: *****  
AWS Secret Access Key: *****  
Default region: ap-south-1  
Output format: json
```

```
aws sts get-caller-identity
```

3. Update kubeconfig

```
aws eks update-kubeconfig --region ap-south-1 --name my-eks-cluster
```

```
kubectl get nodes
```

4. Authenticate Docker to Amazon ECR

```
aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin 123456789012.dkr.ecr.ap-south-1.amazonaws.com
```

5. Verify the Image Exists

```
aws ecr describe-images --repository-name my-app
```

6. (Optional) Create an Image Pull Secret

```
kubectl create secret docker-registry ecr-secret --docker-  
server=123456789012.dkr.ecr.ap-south-1.amazonaws.com --docker-  
username=AWS --docker-password=$(aws ecr get-login-password --region  
ap-south-1)
```

```
kubectl get secret
```

7. Create deployment.yaml

```
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: my-app  
spec:  
  replicas: 2  
  selector:  
    matchLabels:  
      app: my-app  
  template:  
    metadata:  
      labels:  
        app: my-app  
    spec:  
      containers:  
        - name: my-app  
          image: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/my-  
app:latest  
          ports:  
            - containerPort: 80  
          imagePullPolicy: Always  
          imagePullSecrets:  
            - name: ecr-secret
```

8. Deploy

```
kubectl apply -f deployment.yaml
```

```
kubectl get deployment
```

9. Check Pods

```
kubectl get pods
```

```
kubectl describe pod <pod-name>
```

```
kubectl logs <pod-name>
```

10. Create service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: my-app-service
spec:
  selector:
    app: my-app
  type: LoadBalancer
  ports:
    - port: 80
      targetPort: 80
```

```
kubectl apply -f service.yaml
```

11. Verify Service

```
kubectl get svc
```

```
kubectl get svc my-app-service
```

Useful Commands

```
kubectl get deployments
kubectl get pods
kubectl get svc
kubectl describe deployment my-app
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl scale deployment my-app --replicas=5
kubectl rollout restart deployment my-app
kubectl rollout status deployment my-app
kubectl rollout undo deployment my-app
kubectl delete deployment my-app
kubectl delete service my-app-service
```

Best Practices

- Prefer IAM Roles for Service Accounts (IRSA) or node IAM roles instead of imagePullSecrets.
- Use immutable image tags instead of latest.
- Configure readiness and liveness probes.
- Define CPU and memory requests/limits.
- Use AWS Load Balancer Controller for ALB-based ingress where appropriate.